Implementação e Análise de Estratégias de Algoritmos para Solução de Problema NP

Ciclo Hamiltoniano Direcionado

Autores

Nome completo	Matrícula
Fabio Firanzi	[**matrícula**]
Finéias Antonio	[**matrícula**]
Heitor Vaz	[**matrícula**]
Tulio Vitorette	[**matrícula**]

Sumário

1 Introdução **3**

1.1 O problema . . . 3

1.2 Dois exemplos mínimos . . . 3

1.3 Aplicações práticas . . . 4

2 Prova: CHD pertence à classe NP **5**

2.1 CHD $\in$ NP . . . 5

2.2 CHD é NP-completo (além do que o edital pede) . . . 5

3 Implementação **6**

3.1 Estrutura comum das implementações . . . 6

3.2 Força bruta . . . 7

3.3 Backtracking . . . 7

3.4 Divisão e conquista: por que não há algoritmo . . . 8

3.5 Programação dinâmica (Held-Karp) . . . 8

3.6 Guloso . . . 9

3.7 Heurística: recozimento simulado . . . 10

3.8 Caso especial: torneios (Camion) . . . 10

4 Análise de complexidade **10**
4.1 Tempo e espaço teóricos 10
4.2 O que o contador mede 12

5 Análise de resultados **12**
5.1 Método 12
5.2 Os três exatos no pior caso 13
5.3 Efeito do filtro de conectividade forte 14
5.4 Variação entre tipos 14
5.5 Guloso e recozimento: quem acha o ciclo? 15
5.6 Camion em torneios 16

6 Conclusão **16**

1 Introdução

1.1 O problema

Seja $G = (V, A)$ um grafo direcionado simples, sem laços, com $n = |V| \geq 2$ vértices. Um *ciclo hamiltoniano direcionado* é uma sequência $(v_1, v_2, \ldots, v_n)$ tal que

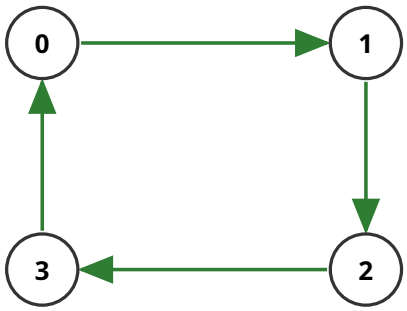
- os v_i são todos distintos e cobrem V;
- $(v_i, v_{i+1}) \in A$ para $i = 1, \ldots, n-1$;
- $(v_n, v_1) \in A$.

O **problema de decisão (CHD)** pergunta: dado G, existe um ciclo hamiltoniano direcionado? O código deste trabalho resolve a *versão de busca*: devolve a sequência quando ela existe. A resposta de decisão é o sim ou o não dessa busca, e a própria sequência é a prova de que a resposta é sim, usada na Seção 2.

Convenção de simetria. Rotacionar um ciclo não cria outro ciclo. Por isso todos os métodos exatos fixam $v_1 = 0$. Inverter o sentido, ao contrário, produz outro ciclo direcionado e não é equivalente: o sentido oposto continua valendo e entra na busca.

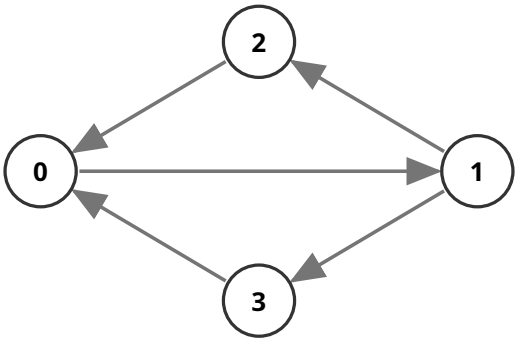
1.2 Dois exemplos mínimos

Exemplo com ciclo. Arcos $(0,1), (1,2), (2,3), (3,0)$. O ciclo é $0 \to 1 \to 2 \to 3 \to 0$, em verde na figura.



Digrafo com ciclo hamiltoniano direcionado.

Exemplo sem ciclo, com o digrafo forte. Arcos $(0,1), (1,2), (2,0), (1,3), (3,0)$. De 3 só se sai para 0, e em 3 só se entra por 1, então 3 teria de ocupar o trecho $1 \to 3 \to 0$. O vértice 2 não cabe nesse trecho com os arcos existentes. O digrafo é fortemente conexo: conectividade forte é condição *necessária* e não suficiente.



Digrafo fortemente conexo sem ciclo hamiltoniano.

Os dois exemplos podem ser executados por qualquer programa do trabalho, com os tipos `ciclo_simples` e `forte_sem_ciclo`.

1.3 Aplicações práticas

Em cada aplicação abaixo, o que é vértice, o que é arco e por que a volta ao início importa. A dificuldade é a mesma nos quatro casos: o ciclo hamiltoniano é NP-completo, e a solução exata só funciona para digrafos pequenos.

Entregas urbanas (Correios, iFood, Amazon). Cada endereço de entrega é um vértice, e um arco é o trecho de rua que leva de um endereço a outro. Como há ruas de mão única, ir de A a B não equivale a ir de B a A, e o digrafo é naturalmente *direcionado*. O entregador parte do depósito, visita cada endereço uma vez e volta ao depósito: a volta ao início é o retorno da van ou da moto à base. As empresas resolvem a versão com custos (caixeiro viajante assimétrico e roteirização de veículos), e o CHD é a pergunta de viabilidade dentro dela: o caso em que todo arco existente custa 1 e o limite é $B = n$. Com centenas de paradas por dia, a solução exata é inviável (Seção 4) e as empresas usam heurísticas, como o recozimento deste trabalho. O texto não atribui a elas razão de aproximação provada.

Computação com DNA (Adleman, 1994). A primeira demonstração de computação com moléculas de DNA resolveu um problema de caminho hamiltoniano num *digrafo* de 7 vértices [1]. Cada cidade foi codificada por uma sequência de DNA e cada arco por uma sequência que liga o fim de uma cidade ao início da seguinte. Misturadas num tubo de ensaio, as moléculas formaram, em paralelo, todos os caminhos possíveis, e as etapas seguintes eliminaram os que não passavam por todas as cidades exatamente uma vez. O experimento é famoso por mostrar a ideia, e também por mostrar o limite: o número de moléculas necessárias cresce com o número de caminhos, isto é, de forma exponencial, e a abordagem não escala.

Montagem de genomas (Projeto Genoma Humano e bactérias). Para sequenciar um genoma, a máquina lê milhões de fragmentos curtos. Na formulação clássica, cada fragmento é um vértice, e um arco liga dois fragmentos quando o final de um se sobrepõe ao começo do outro. Uma sequência que usa cada fragmento uma vez é um

caminho hamiltoniano; no genoma circular de uma bactéria ou de um plasmídeo é um ciclo hamiltoniano, e a volta ao início é a própria circularidade do DNA. Como o problema é NP-completo, os montadores modernos mudaram de modelo: usam o grafo de de Bruijn e o caminho euleriano, que é polinomial [5]. O problema deste trabalho explica uma mudança real de abordagem numa área muito conhecida.

Campeonato por pontos corridos (Brasileirão, xadrez). Num campeonato em que todos jogam contra todos, os competidores são vértices e uma vitória é um arco do vencedor para o perdedor. Todo par tem exatamente um sentido (sem empate, ou desconsiderando-o), então o digrafo é um torneio. Se ele é fortemente conexo, o teorema de Camion [4] garante um ciclo hamiltoniano: uma ordenação circular em que cada competidor venceu o seguinte, e o último venceu o primeiro. O ciclo explica por que o "ganhou de" não é transitivo (A vence B, B vence C, C vence A). Este é o caso em que a pergunta deixa de ser NP-completa: a construção da Seção 3.8 acha o ciclo em tempo polinomial, e mostra que a dificuldade mora na estrutura do digrafo e não no nome do problema.

2 Prova: CHD pertence à classe NP

2.1 CHD $\in$ NP

Um problema está em NP quando, se alguém propõe uma resposta "sim", dá para conferi-la rapidamente, em tempo polinomial [6, 12]. Não é preciso saber *achar* a resposta, só *conferir*.

No CHD, a resposta "sim" proposta é uma sequência de vértices, $c = (c_1, \ldots, c_n)$. Para conferir se ela é um ciclo hamiltoniano direcionado (função `verifica_ciclo`), basta checar três coisas:

1. A sequência tem exatamente n vértices.
2. Nenhum vértice se repete. Um vetor de n marcas resolve em n passos. Como são n vértices diferentes em n posições, nenhum vértice fica de fora.
3. Cada vértice tem arco para o próximo, e o último tem arco para o primeiro. São n consultas à matriz, cada uma em tempo constante.

Ao todo são cerca de n passos, um tempo polinomial. A conferência aceita se, e somente se, c é de fato um ciclo hamiltoniano direcionado de G. Logo, CHD está em NP.

2.2 CHD é NP-completo (além do que o edital pede)

Um problema é NP-completo quando está em NP e é pelo menos tão difícil quanto os problemas mais difíceis de NP. Para mostrar isso, usa-se um problema que já se sabe ser difícil: o ciclo hamiltoniano em grafos *sem direção*, que é NP-completo [10, 8].

A ideia é transformar um problema no outro:

1. Pegue um grafo sem direção e troque cada aresta entre u e v por dois arcos, um de u para v e outro de v para u. Isso gera um digrafo.

2. Se o grafo tem um ciclo hamiltoniano, o digrafo tem o mesmo ciclo, pois todo caminho existe nos dois sentidos.

3. Se o digrafo tem um ciclo hamiltoniano, o grafo também tem, pois cada arco usado veio de uma aresta.

Logo, quem soubesse resolver o CHD saberia resolver o ciclo hamiltoniano sem direção, que é um problema difícil. Então o CHD também é difícil e, como já está em NP, é NP-completo (problema GT38 de [8]).

3 Implementação

3.1 Estrutura comum das implementações

Cada estratégia é um programa independente, com a sua própria função `main`. Todos usam as mesmas peças, descritas abaixo, para que as comparações sejam justas: o digrafo, o contador e o verificador são iguais para todos.

Organização dos arquivos. Há uma pasta para a parte comum e uma para cada estratégia. Cada pasta de estratégia tem o arquivo com o algoritmo (por exemplo, `forca_bruta.cpp`) e o seu `main.cpp`, que apenas prepara o digrafo, chama o algoritmo e mostra o resultado. O comando `make` compila todos os programas.

O digrafo. É guardado como uma tabela $n \times n$, em que a posição (u, v) indica se existe o arco de u para v. Assim, perguntar se um arco existe custa um passo, o que importa porque todos os algoritmos fazem essa pergunta muitas vezes. A tabela ocupa n^2 posições. Não existem laços (arcos de um vértice para ele mesmo): se alguém tenta adicionar um, ele é ignorado. A tabela distingue os sentidos, então a existência de $u \to v$ não implica a de $v \to u$.

O contador de operações. Para a análise por contagem de instruções, o digrafo conta toda consulta de arco que recebe, e cada algoritmo conta também a sua unidade de trabalho principal. São três contadores:

- `consultas_arco`: quantas vezes se perguntou se um arco existe. Todos os algoritmos usam.
- `permutacoes`: quantas ordens de vértices foram testadas. Só a força bruta usa.
- `nos`: a unidade de trabalho dos demais algoritmos, que muda de um para outro: chamadas da busca no backtracking, estados expandidos no Held-Karp, escolhas no guloso e movimentos no recozimento.

Os números de algoritmos diferentes não são somados em um total único, porque um estado do Held-Karp e uma permutação da força bruta não são a mesma operação.

O verificador. A função `verifica_ciclo` recebe um digrafo e uma sequência de vértices e responde se ela é um ciclo hamiltoniano direcionado: confere se tem n vértices, se nenhum se repete e se cada vértice tem arco para o seguinte, inclusive do último para o primeiro. É o verificador da prova de NP da seção anterior. Todos os programas o usam para conferir o ciclo que o algoritmo devolveu, antes de mostrá-lo.

O gerador de digrafos. Cria os digrafos de teste a partir de um tipo e do tamanho n:

Tipo	O que gera
`ciclo_simples`	Só o ciclo $0 \to 1 \to \cdots \to (n-1) \to 0$. Tem ciclo hamiltoniano, e é o único.
`forte_sem_ciclo`	O exemplo fixo de 4 vértices da introdução: fortemente conexo e sem ciclo hamiltoniano. Ignora o n.
`pior_caso`	Um digrafo completo sobre $n-1$ vértices, mais um vértice ligado só ao vértice 1. Não tem ciclo, e obriga os métodos exatos a esgotar a busca.
`aleatorio`	Cada arco existe com probabilidade 0,35. Pode ter ou não ter ciclo, e a chance de ter cresce com n.
`armadilha_indice`	Digrafo de 4 vértices em que o guloso por índice falha, embora exista ciclo.
`armadilha_warnsdorff`	Digrafo de 4 vértices em que o guloso de Warnsdorff falha, embora exista ciclo.

O tipo `aleatorio` usa uma semente. Sem indicação, ela é sorteada a cada execução, e o programa imprime a semente usada. Com a opção `--semente=N` o mesmo digrafo é gerado de novo, o que permite rodar dois algoritmos no mesmo digrafo e comparar as respostas.

O programa de cada estratégia. Todos funcionam do mesmo jeito: recebem o tipo e o tamanho n pela linha de comando, geram o digrafo, rodam o algoritmo e imprimem o ciclo (ou a informação de que não foi encontrado), os contadores, o tempo e a semente do digrafo. O tempo é medido com `std::chrono::steady_clock` e cobre só a execução do algoritmo, sem a geração do digrafo.

3.2 Força bruta

Fixa o vértice 0 na origem e gera as $(n-1)!$ permutações dos demais com `std::next_permutation`. Para cada permutação consulta os n arcos do ciclo e para na primeira que fecha. Não há poda: poda é o que distingue o backtracking na comparação.

3.3 Backtracking

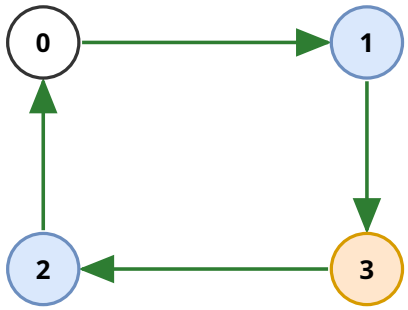
Busca em profundidade a partir de 0. O próximo vértice é um vizinho de saída ainda fora do caminho; com n vértices no caminho, o único teste é o arco de volta a 0. Duas ordens de vizinhos (mesma resposta, árvores de tamanhos diferentes): índice crescente e a regra de Warnsdorff adaptada (primeiro o vizinho com menos saídas livres, ideia herdada do passeio do cavalo, sem herdar uma prova). A ordenação é detalhe do backtracking,

não um algoritmo guloso. O parâmetro `usar_filtro` permite a ablação da poda por conectividade forte.

3.4 Divisão e conquista: por que não há algoritmo

Divisão e conquista, no sentido de [6], parte a entrada em subproblemas *independentes*, resolve cada um e combina as respostas. No CHD a combinação não é independente: um ciclo hamiltoniano de G não é a união de ciclos hamiltonianos dos subdigrafos induzidos por uma partição $V = L \cup R$, porque o ciclo pode alternar entre L e R várias vezes.

O atalho que *parece* divisão (permutar um bloco L, permutar o outro bloco R e concatenar) responde outra pergunta. Contraexemplo, com origem 0, $L = \{1, 2\}$ e $R = \{3\}$: as concatenações possíveis são só $(0, 1, 2, 3)$, $(0, 2, 1, 3)$, $(0, 3, 1, 2)$ e $(0, 3, 2, 1)$. O digrafo de arcos $(0, 1), (1, 3), (3, 2), (2, 0)$ tem o ciclo $0 \to 1 \to 3 \to 2 \to 0$ e nenhuma das quatro sequências, então o método de blocos responde *não* num digrafo que *tem* ciclo.



O ciclo $0 \to 1 \to 3 \to 2 \to 0$ intercala os blocos $L = \{1, 2\}$ (azul) e $R = \{3\}$ (laranja), e nenhuma concatenação de blocos o produz.

Por isso o grupo **não apresenta** esse atalho como solução. Ele existiu apenas para demonstrar o falso negativo, que o programa `ciclo_por_blocos` exibe com o tipo `ciclo_intercalado`.

A decomposição que funciona usa outro subproblema: "caminho simples que começa em 0, cobre o conjunto S e termina em v". Esses subproblemas se repetem ao longo da busca; resolver cada um uma vez e reutilizar é programação dinâmica, a próxima subseção. Algoritmos mais recentes (por exemplo, a inclusão-exclusão de Björklund para o caso não direcionado [3]) ficam fora do código, e o algoritmo exato de referência do trabalho é Held-Karp.

3.5 Programação dinâmica (Held-Karp)

Estado: $\mathrm{dp}[S][v]$ é verdadeiro quando existe caminho simples que começa em 0, visita exatamente o conjunto S e termina em v, com $0 \in S$ e $v \in S$. Base: $\mathrm{dp}[\{0\}][0] =$ verdadeiro. Transição: $\mathrm{dp}[S][v]$ é verdadeiro quando existe $u \in S \setminus \{v\}$ com $\mathrm{dp}[S \setminus \{v\}][u]$ verdadeiro e arco (u, v). Há ciclo quando existe $v \neq 0$ com $\mathrm{dp}[V][v]$ verdadeiro e arco $(v, 0)$ [9, 2].

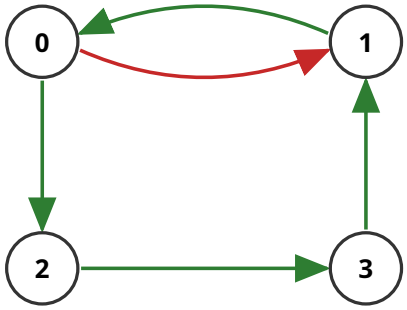
Cada conjunto é uma máscara de bits, processada em ordem crescente (a máscara sem o bit de v é menor e já foi calculada). Para cada máscara guarda-se um conjunto de

vértices finais possíveis (um inteiro de 32 bits) e, para cada estado, o predecessor, de modo que o ciclo é reconstruído, e não só o booleano. A função aceita até $n = 22$ e, acima disso, lança erro em vez de responder que o ciclo não existe.

3.6 Guloso

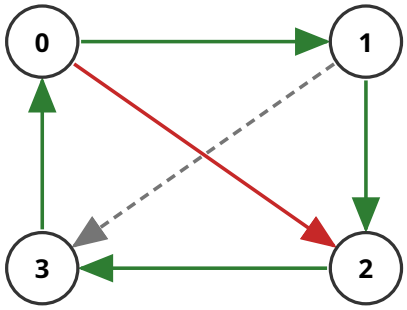
Dois critérios, mesma estrutura: parte de 0 e, enquanto houver vértice livre, escolhe um vizinho de saída livre; se empaca antes de cobrir V, ou se o último não aponta para 0, devolve falha. O critério `indice` escolhe o menor índice; o critério `warnsdorff` escolhe o vizinho com menos saídas livres. **Nenhum dos dois tem garantia**, e cada um tem contraexemplo automatizado:

Critério `indice`. Arcos $(0,1), (0,2), (2,3), (3,1), (1,0)$. O ciclo $0 \to 2 \to 3 \to 1 \to 0$ existe (verde). O guloso sai de 0 para 1 (vermelho), e de 1 só há arco para 0, que já está no caminho: falha num digrafo que tem ciclo.



Armadilha do critério por índice. Verde: ciclo existente. Vermelho: escolha do guloso.

Critério `warnsdorff`. Arcos $(0,1), (0,2), (1,2), (1,3), (2,3), (3,0)$. O ciclo $0 \to 1 \to 2 \to 3 \to 0$ existe (verde). Saindo de 0, o vértice 1 tem duas saídas livres e o 2 tem uma; a regra escolhe 2 (vermelho), segue a 3 e empaca com 1 de fora.



Armadilha do critério de Warnsdorff. Verde: ciclo existente. Vermelho: escolha do guloso. Tracejado: arco não usado.

Os dois são heurísticas gulosas, sem garantia de otimalidade e sem razão de aproximação.

3.7 Heurística: recozimento simulado

Reformulação de otimização usada pela heurística: entre as permutações com origem 0, maximizar o número de arcos presentes nas n posições do ciclo. O *fitness* vai de 0 a n, e o digrafo tem ciclo exatamente quando o máximo vale n. Essa reformulação permite busca local; **ela não cria razão de aproximação**.

Movimento: retirar um vértice e inseri-lo em outra posição (relocação). A *variação* do fitness depende só dos arcos da fronteira da posição antiga e da nova, e é calculada em $O(1)$. Aceita melhora sempre e aceita piora com probabilidade $e^{\Delta/T}$; esfriamento $T \leftarrow \alpha T$; vários recomeços a partir de permutações aleatórias; para ao achar fitness n ou ao gastar o orçamento [11].

Parâmetros declarados antes da bateria e congelados: $T_0 = 1$, $\alpha = 0{,}995$, $200 \cdot n$ movimentos por recomeço e 10 recomeços. O resultado separa `Ciclo` (ciclo válido encontrado) de `Orcamento` (orçamento esgotado). Esgotar o orçamento **não** prova que o ciclo não existe.

3.8 Caso especial: torneios (Camion)

Teorema 1 (Camion, 1959 [4]) *Um torneio com $n \geq 3$ vértices tem ciclo hamiltoniano direcionado se e somente se é fortemente conexo.*

Este módulo não é uma das seis estratégias do caso geral: mostra a fronteira da dificuldade, pois a mesma pergunta, restrita a torneios fortes, cai em P. Construção: (i) caminho hamiltoniano por inserção; (ii) um arco de volta ao início do caminho fecha um ciclo que cobre um prefixo; (iii) cada vértice de fora que encaixa entre dois consecutivos do ciclo entra ali; (iv) quando nenhum encaixa sozinho, os vértices de fora se dividem entre os que apontam para o ciclo inteiro e os que recebem arco de todo o ciclo, e um arco do segundo grupo para o primeiro permite inserir um par. O módulo recusa o digrafo que não é torneio.

4 Análise de complexidade

4.1 Tempo e espaço teóricos

Usa-se $n = |V|$ e matriz de adjacência, com consulta de arco em $O(1)$.

Função	Tempo	Espaço extra	Garantia
`verifica_ciclo`	$O(n)$	$O(n)$	exata
Força bruta	$\Theta(n \cdot (n-1)!)$ no pior caso	$O(n)$	exata
Backtracking	$O(n!)$ no pior caso; depende da densidade	$O(n)$ (pilha)	exata
Held-Karp	$\Theta(n^2\, 2^n)$	$\Theta(n\, 2^n)$	exata
Guloso, índice	$O(n^2 \log n)$	$O(n)$	nenhuma
Guloso, Warnsdorff	$O(n^3)$	$O(n)$	nenhuma
Recozimento	$O(r \cdot c\, n^2)$, r recomeços, $c = 200$	$O(n)$	nenhuma
Camion (só torneio)	$O(n^3)$	$O(n)$	exata
Blocos contíguos	$O(n! \cdot n)$ $(n \leq 10)$	$O(n)$	incorreto

Força bruta. Gera $(n-1)!$ permutações; cada uma consulta até n arcos e copia n posições. O melhor caso termina na primeira permutação; o pior caso é $\Theta(n \cdot (n-1)!)$, e é o caso dos digrafos sem ciclo. Espaço $O(n)$, pois `next_permutation` gera no lugar.

Backtracking. A árvore tem profundidade n e grau no máximo $n-1$, portanto $O(n!)$ nós no pior caso. Com a poda de vértice já usado, o número de nós em um digrafo completo sobre m vértices é $\sum_k m!/(m-k)! \approx e \cdot m!$. Para o espaço, a pilha de recursão e o vetor de usados são $O(n)$.

Held-Karp. São 2^n máscaras, para cada uma até n vértices finais e para cada vértice até n vizinhos de saída: $\Theta(n^2\, 2^n)$. O tempo depende de n e não do número de arcos, exceto pelas podas por máscara inalcançável. O espaço é dominado pela tabela de predecessores, com $n\, 2^n$ inteiros de 4 bytes, mais 2^n palavras de 32 bits do conjunto de finais. Para $n = 22$ são $22 \cdot 2^{22} \cdot 4 \approx 369$ MB mais 16 MB, aproximadamente 385 MB. Por isso a memória limita o n *antes* do tempo, e o código recusa $n > 22$.

Guloso. Cada passo percorre os vizinhos de saída ($O(n)$) e ordena os candidatos ($O(n \log n)$); são $n-1$ passos, daí $O(n^2 \log n)$. No critério de Warnsdorff, cada candidato conta suas próprias saídas livres ($O(n)$), o que leva a $O(n^3)$. (Trocar a ordenação por uma busca do mínimo reduziria para $O(n^2)$ e $O(n^3)$ sem mudar a resposta; não foi feito porque o objetivo do trabalho é a comparação.)

Recozimento. O cálculo da variação é $O(1)$ (seis consultas de arco), mas aplicar a relocação aceita desloca até n posições, então um movimento custa $O(n)$ quando aceito. Com r recomeços de $c\, n$ movimentos, o tempo é $O(r\, c\, n^2)$, proporcional ao orçamento declarado. Espaço $O(n)$.

Camion. O caminho por inserção custa $O(n^2)$ (cada vértice varre o caminho). Cada rodada de inserção percorre os vértices de fora e o ciclo, $O(n^2)$, e pode haver até n rodadas, portanto $O(n^3)$ no pior caso. Para $n \leq 30$ no experimento o tempo é irrelevante frente aos métodos exponenciais.

4.2 O que o contador mede

A contagem de instruções usa as três grandezas do `Contador`. A *unidade comum* de comparação entre estratégias é o tempo de parede. Cada algoritmo conta "nós" de uma natureza distinta, e por isso o texto não soma contagens de algoritmos diferentes num "total de instruções" único.

Algoritmo	O que é um "nó"
Força bruta	uma permutação testada (valor medido: exatamente $(n-1)!$ no pior caso)
Backtracking	uma chamada recursiva da busca
Held-Karp	um estado (máscara, vértice final) expandido
Guloso	uma escolha de vizinho
Recozimento	um movimento de relocação proposto
Camion	uma inserção

5 Análise de resultados

5.1 Método

Digrafos de teste. Semente base 20261107. Seis tipos de digrafos, cada um com gerador próprio:

- **Ciclo + arcos extras**: um ciclo plantado mais n arcos aleatórios (resposta sim).
- **DAG aleatório**: arcos só para a frente, $p = 0{,}5$ (resposta não; não é fortemente conexo).
- **Erdős–Rényi direcionado**, com $p = 0{,}35$ fixo e com $p = (\ln n)/n$, a ordem do limiar de aparecimento de grau mínimo de entrada e saída 1 [7]. Uma amostra finita não é tratada como demonstração do teorema.
- **Torneio aleatório**: sim quando é forte.
- **Pior caso (sem ciclo)**: um digrafo completo sobre $n-1$ vértices mais um vértice x ligado apenas ao vértice 1 (arcos $1 \to x$ e $x \to 1$). O digrafo é fortemente conexo, não tem ciclo hamiltoniano (passar por x exige usar 1 duas vezes) e obriga os métodos exatos a esgotar a busca.

Juiz. Até $n = 14$ o Held-Karp julga os demais; até $n = 11$ a força bruta concorda com ele (nos 288 digrafos em que mais de um método exato rodou, as respostas coincidiram). Todo ciclo devolvido passa por `verifica_ciclo`. Acima do alcance da PD, "ciclo válido encontrado" ou "orçamento esgotado"; a palavra "inexistente" fica reservada aos métodos exatos.

Execução. AMD Ryzen 7 5700G, 31 GiB de memória, g++ 16.2 com `-O2`, um núcleo. Foram 2528 medições, com 8 digrafos por tamanho e tipo. Timeout de 30 s por chamada exata: 0 medições foram censuradas. Tamanhos: força bruta e backtracking $n = 6$ a 11;

Held-Karp $n = 6$ a 14; guloso, recozimento e Camion até $n = 30$. Os números de nós são reproduzíveis com os programas do trabalho, pois a semente é fixa; os tempos variam um pouco de uma execução para outra.

Cuidado com a resolução do relógio. Tempos abaixo de aproximadamente 0,005 ms são ruído de medição (resolução de 0,001 ms nos dados) e não são interpretados. A discussão se apoia em tempos acima de 0,01 ms e nas contagens de nós, que não têm ruído.

5.2 Os três exatos no pior caso

	Força bruta		Backtracking		Held-Karp	
n	nós	ms	nós	ms	nós	ms
6	120	0.002	81	0.002	41	0.001
7	720	0.009	391	0.010	97	0.001
8	5040	0.075	2283	0.053	225	0.003
9	40320	0.751	15657	0.380	513	0.005
10	362880	6.029	123301	3.049	1153	0.026
11	3628800	57.604	1096011	38.012	2561	0.070
12	–	–	–	–	5633	0.150
13	–	–	–	–	12289	0.385
14	–	–	–	–	26625	0.955

A Figura 1 e a Figura 2 mostram os mesmos dados em escala logarítmica.

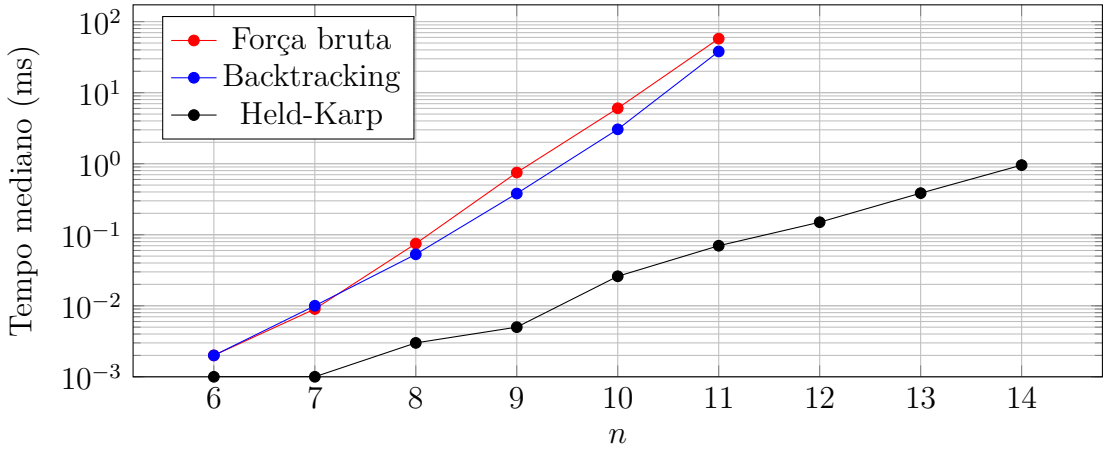


Figura 1: Tempo mediano contra n no tipo pior caso (escala logarítmica).

Força bruta. Os nós medidos são exatamente $(n-1)!$: 120 em $n = 6$ e 3 628 800 em $n = 11$, igual à teoria. O tempo cresce por um fator de cerca de 9,6 entre $n = 10$ e $n = 11$ (6,03 → 57,6 ms), compatível com o fator $n - 1 = 10$.

Backtracking. A poda de vértice usado reduz os nós a cerca de 30% da força bruta em $n = 11$ (1 096 011 contra 3 628 800), valor próximo de $e \cdot 9! \approx 986\,000$ da fórmula da seção

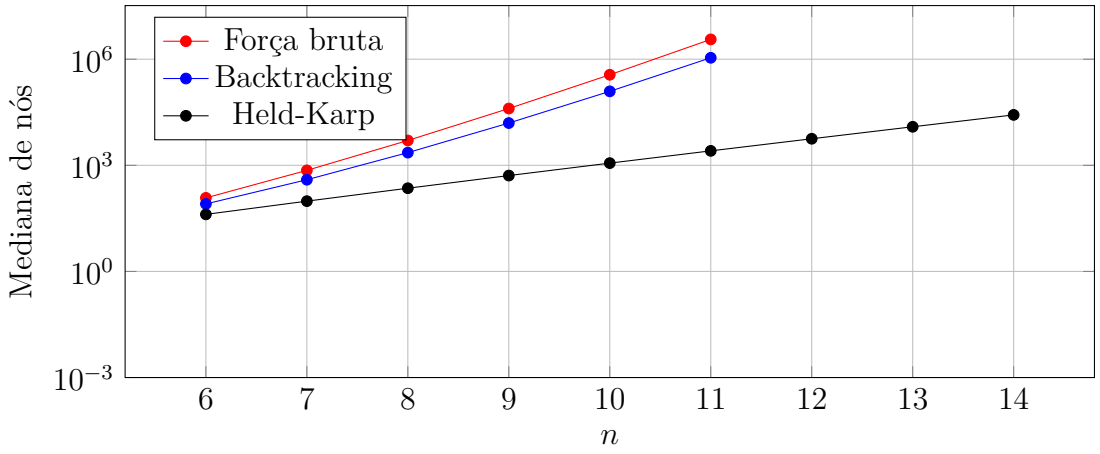


Figura 2: Mediana de nós contra n no tipo pior caso (escala logarítmica).

anterior. O ganho é de uma constante, não da ordem: a razão entre nós consecutivos continua a crescer com n. Neste tipo as duas ordens de vizinhos têm o mesmo número de nós (esperado: o grafo é completo sobre $n-1$ vértices e o digrafo não tem ciclo, logo a árvore é esgotada de qualquer modo); a ordem de Warnsdorff custa mais tempo pelo cálculo da chave (43,6 contra 38,0 ms).

Held-Karp. Os nós crescem por um fator de aproximadamente 2,2 por unidade de n ($2561 \to 5633 \to 12289 \to 26625$), compatível com $\Theta(n^2\,2^n)$, e o tempo cresce por cerca de 2,5 por unidade de n (0,150 → 0,385 → 0,955 ms). Em $n = 11$, Held-Karp leva 0,070 ms contra 57,6 ms da força bruta, uma razão de cerca de 800 vezes, e a razão aumenta com n. O cruzamento entre o fatorial e $n^2\,2^n$ acontece já em $n \approx 7$, onde as duas curvas são indistinguíveis dentro do ruído; a partir daí a diferença é de ordem.

5.3 Efeito do filtro de conectividade forte

No tipo DAG a resposta é sempre não. Com o filtro, o backtracking responde sem expandir nenhum nó; sem o filtro, expande alguns:

n	nós com filtro	nós sem filtro
6	0	6
7	0	16
8	0	14
9	0	15
10	0	44
11	0	31

O filtro corta a árvore inteira, mas é *barato e correto*, e não resolve o caso forte: no tipo pior caso, que passa no filtro, a busca esgota 1,1 milhão de nós em $n = 11$.

5.4 Variação entre tipos

Tipo ($n = 11$)	Força bruta		Backtracking (Warnsdorff)		Held-Karp	
	mediana	máximo	mediana	máximo	mediana	máximo
Ciclo + arcos extras	0.000	0.000	0.001	0.002	0.004	0.049
DAG aleatório	0.000	0.000	0.000	0.000	0.000	0.001
Erdős–Rényi, $p = 0{,}35$	3.131	36.111	0.005	0.030	0.016	0.044
Erdős–Rényi, $p = \ln n/n$	0.000	0.000	0.000	0.000	0.001	0.001
Torneio aleatório	0.822	14.243	0.002	0.039	0.058	0.076

Tempos em milissegundos, $n = 11$, oito digrafos por tipo.

Nos tipos em que existe ciclo e há muitos arcos, o backtracking acha rapidamente (cauda curta). A força bruta tem cauda longa nos tipos com ciclo: em Erdős–Rényi com $p = 0{,}35$ a mediana é de 3,1 ms e o máximo, 36 ms, pois a primeira permutação que fecha pode estar tarde na ordem lexicográfica. O Held-Karp é estável (o tempo muda pouco entre tipos do mesmo n nos que existe ciclo), como esperado de uma complexidade que não depende da densidade. Mediana e máximo contam histórias diferentes, e os dois são publicados.

5.5 Guloso e recozimento: quem acha o ciclo?

Tipo	Digrafos com ciclo	Guloso índice	Guloso Warnsdorff	Recozimento
Ciclo + arcos extras	72	48/48	10/48	47/48
DAG aleatório	0	–	–	–
Erdős–Rényi, $p = 0{,}35$	41	1/27	0/27	27/27
Erdős–Rényi, $p = \ln n/n$	3	0/3	0/3	3/3
Torneio aleatório	62	6/42	3/42	42/42
Pior caso (sem ciclo)	0	–	–	–

Digrafos em que o Held-Karp (juiz) confirma ciclo, $n = 6$ a 14. A segunda coluna é o total de digrafos com ciclo; as heurísticas rodam só em parte dos tamanhos, então cada célula é encontrados / digrafos com ciclo *em que a heurística rodou*.

- **O guloso por índice acerta 48 de 48 no tipo ciclo + arcos extras por um efeito do gerador**: o ciclo plantado é $0 \to 1 \to 2 \to \cdots$, que é exatamente a ordem de índice. Isso mede o viés do tipo, e não a qualidade do critério; nos demais tipos (Erdős–Rényi, torneio) o mesmo critério encontra 1/27 e 6/42.
- **A regra de Warnsdorff** é pior que o índice nesses digrafos (10/48, 0/27, 3/42). O contraexemplo da Seção 3.6 mostra que é falha de projeto, e não azar.
- **O recozimento** encontra 47/48, 27/27, 3/3 e 42/42 dos ciclos, recuperando a maior parte das falhas dos gulosos, no mesmo orçamento fixo. Mas continua sem como provar que o ciclo não existe: nos digrafos sem ciclo (DAG, pior caso) ele sempre termina por `Orcamento`, e isso não prova nada.
- O tipo de limiar $p = (\ln n)/n$ tem só 3 digrafos com ciclo no total. A amostra é pequena e não sustenta conclusão estatística sobre o limiar.

5.6 Camion em torneios

Em torneios fortes, o algoritmo de Camion devolve o ciclo em tempo desprezível (0,003 ms em $n = 30$, 29 nós, um por vértice) enquanto o Held-Karp em $n = 14$ já gasta cerca de 0,9 ms e 34 827 nós. O mesmo problema, restrito a torneios, é polinomial; isso é o que a seção de aplicações chamou de fronteira da dificuldade.

6 Conclusão

O trabalho implementa, com contagem de instruções e medição de tempo, cinco das seis estratégias pedidas (força bruta, backtracking, programação dinâmica, guloso e recozimento como heurística) e justifica por escrito por que divisão e conquista não se sustenta no caso geral. O resultado experimental principal é a separação por ordem de grandeza: força bruta cresce com $(n-1)!$ nós medidos, backtracking diminui a constante e continua fatorial, e Held-Karp cresce com 2^n, com razão de cerca de 800 vezes em $n = 11$.

Limites. O teto prático do Held-Karp é a memória, não o tempo: a tabela de predecessores chega a cerca de 385 MB em $n = 22$. O guloso não tem garantia e erra nos contraexemplos da Seção 3.6. O recozimento recupera quase todos os ciclos nos testes, mas nunca prova inexistência, e por isso nunca decreta "não existe": ele separa "ciclo válido" de "orçamento esgotado". Os tempos abaixo de 0,005 ms foram descartados por resolução do relógio, e o tipo de limiar teve poucos digrafos com ciclo.

Dificuldades. (i) A reconstrução do ciclo no Held-Karp exige guardar o predecessor de cada estado, o que multiplica a memória por n. (ii) A tentação de apresentar o particionamento em blocos como divisão e conquista; o contraexemplo mostrou o falso negativo, e o trabalho o expõe em vez de o esconder. (iii) O tipo aleatório ingênuo dá digrafos ou triviais (DAG) ou fáceis (arcos extras), e a comparação entre os exatos só ficou informativa com o tipo pior caso. (iv) Distinguir, em todos os métodos, "não encontrou" de "não existe", para não afirmar o que a heurística não prova.

Referências

[1] L. M. Adleman. Molecular computation of solutions to combinatorial problems. *Science*, 266(5187):1021–1024, 1994.

[2] R. Bellman. Dynamic programming treatment of the travelling salesman problem. *Journal of the ACM*, 9(1):61–63, 1962.

[3] A. Björklund. Determinant sums for undirected Hamiltonicity. In *Proc. 51st IEEE FOCS*, 2010.

[4] P. Camion. Chemins et circuits hamiltoniens des graphes complets. *Comptes Rendus de l'Académie des Sciences*, 249:2151–2152, 1959.

[5] P. E. C. Compeau, P. A. Pevzner, G. Tesler. How to apply de Bruijn graphs to genome assembly. *Nature Biotechnology*, 29:987–991, 2011.

[6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*. 3. ed., MIT Press/McGraw-Hill, 2009.

[7] A. M. Frieze. An algorithm for finding Hamilton cycles in random directed graphs. *Journal of Algorithms*, 9(2):181–204, 1988.

[8] M. R. Garey, D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[9] M. Held, R. M. Karp. A dynamic programming approach to sequencing problems. *Journal of the SIAM*, 10(1):196–210, 1962.

[10] R. M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, Plenum, 1972, pp. 85–103.

[11] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[12] N. Ziviani. *Projeto de Algoritmos com implementações em Java e C++*. Cengage Learning (Thomson/Pioneira), São Paulo, 1. ed., 2006.